

Retrieval Augmented Generation

Alexandra Birch



Based on slides by Pasquale Minervini

Outline

Aim: Understand how machines learn to read texts and answer questions

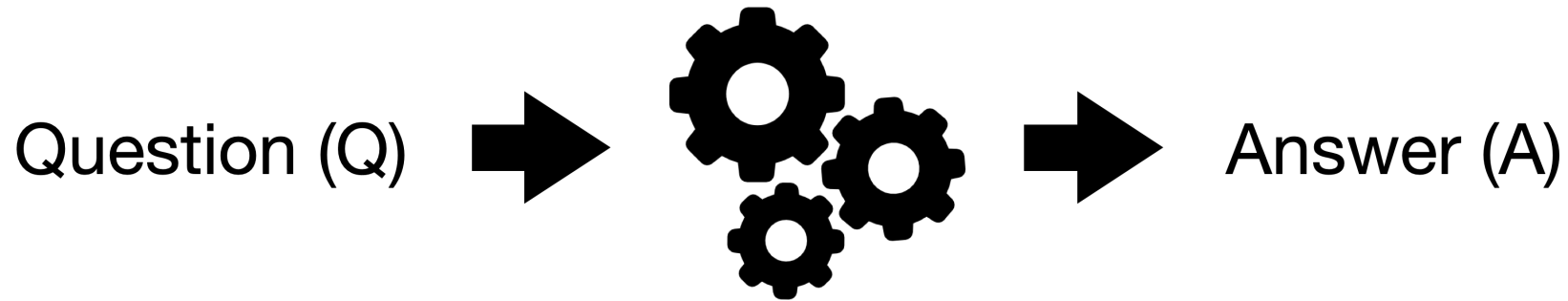
- Recap
- Retriever-Reader Architecture
- Retrievers
- RAG
- Advanced RAG
- Summary

Reading:

- Jurafsky and Martin 3rd Edition pdf August 2025 [Chapter 11](#)

Recap

Open Domain QA



Open-Domain Question Answering (ODQA):

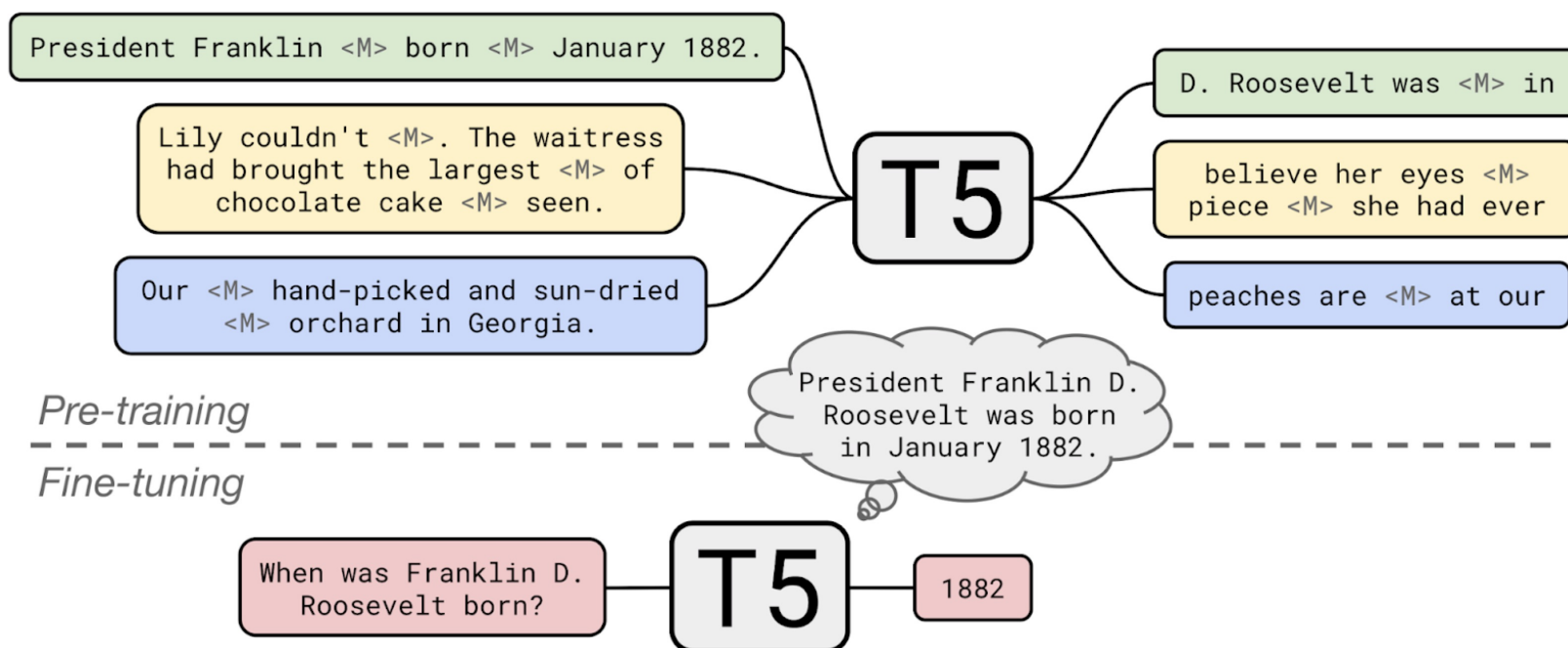
We do not assume we are given a passage together with the question

We can only access a large collection of documents (e.g., Wikipedia) — we don't know which document contains the answer, and the goal is to answer any open-domain questions.

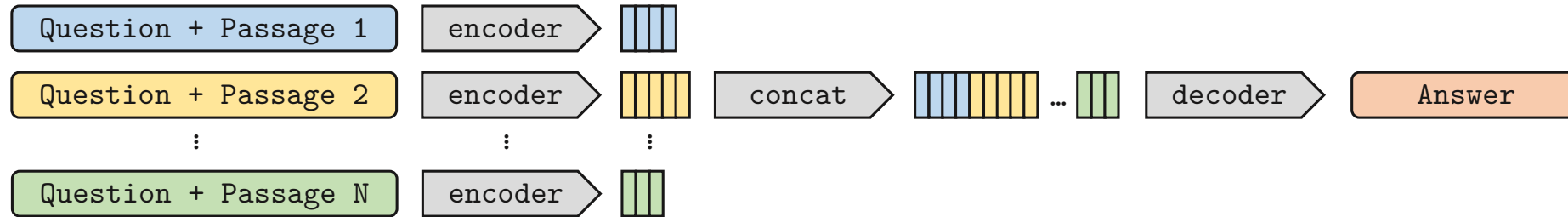
Both more challenging and more practical/useful!

LLMs Can Do Open-Domain QA

LLMs — without an (explicit) retrieval component — can be used to solve Open-Domain Question Answering tasks; knowledge about the world is encoded in their parameters and activations, rather than in a corpus:



Fusion in Decoder



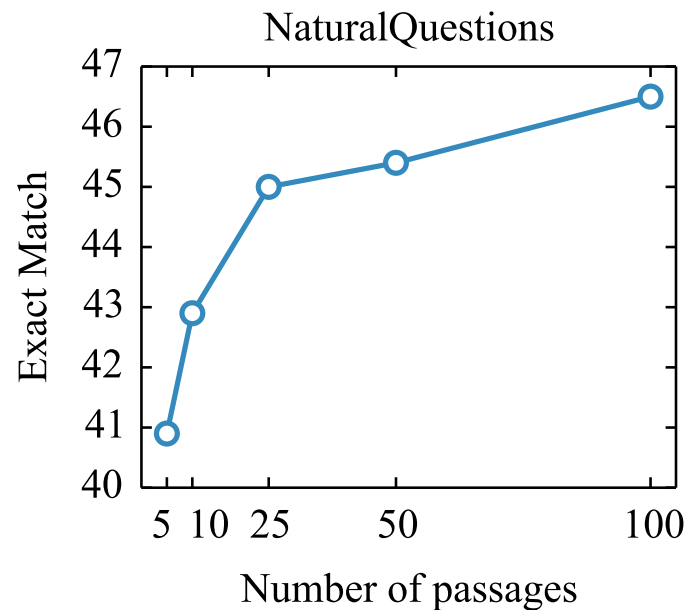
- Dense passage retrieval DPR: passage and question as dense vectors computed by BERT, then rank according to dot product between them. Retrieval using approximate nearest neighbours with the FAISS library
- These are processed independently in the encoder (T5 model) with added tokens "question:", "title:", "context:"
- Decoder attends over the concatenation of all retrieved passages: fusion in the decoder. Can better aggregate evidence from multiple passages
- Computation time grows linearly with number of passages, not quadratically

Izacard and Grave (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering, EACL

Fusion in Decoder

Exact match: discard answers > 5 words, normalize lowercase, remove punctuation & articles

Q: what could problems be with this evaluation?



T5 (Roberts et al., 2020)	36.6	
GPT-3 few shot (Brown et al., 2020)	29.9	175B
Fusion-in-Decoder (base)	48.2	220M
Fusion-in-Decoder (large)	51.4	770M

Foundation for modern RAG and follow-up work

Izacard and Grave (2021). Leveraging Passage Retrieval with Generative Models for Open Domain Question Answering, EACL

Retriever-Reader Architecture

LLMs and their Limitations

LLMs are Extremely Impressive —

- ✓ They can store vast amounts of knowledge in their parameters/activations
- ✓ Very strong results on many tasks, even in few-shot learning settings
- ✓ Very flexible — applicable on a variety of tasks

Input: List the top five US states with the highest per-capita GDP, in order.

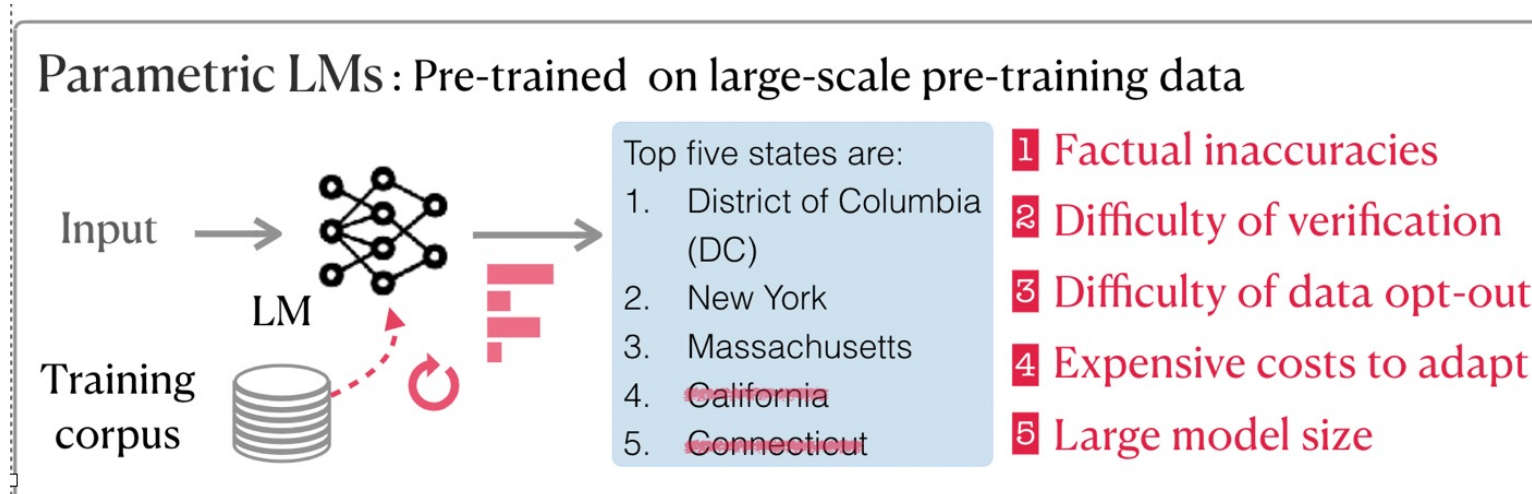


Figure from Asai et al. (2024). Reliable, Adaptable, and Attributable Language Models with Retrieval

Retrieval Augmented Generation

Input: List the top five US states with the highest per-capita GDP, in order.

Retrieval-augmented LMs : Incorporate data at inference



Figure from Asai et al. (2024). Reliable, Adaptable, and Attributable Language Models with Retrieval

Retriever-Reader Architecture

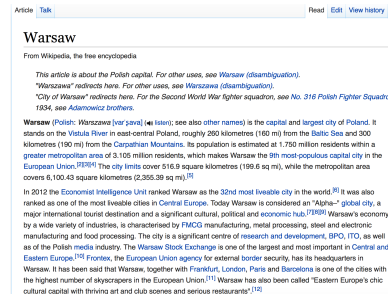
Open-domain QA

SQuAD, TREC, WebQuestions, WikiMovies

Q: How many of Warsaw's inhabitants spoke Polish in 1933?



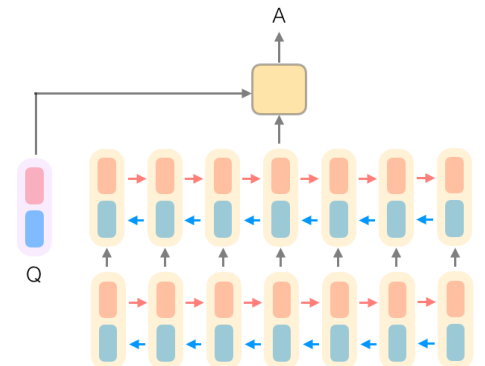
**Document
Retriever**



**Document
Reader**

833,500

Extracted from document



Overview of simple question answering system

Figure from Chen et al. (2017). Reading wikipedia to answer open-domain questions

The Retriever-Reader Framework

Input: a large collection of documents $\mathcal{D} = \{D_1, \dots, D_n\}$ and a question Q

Output: an answer A retrieved from $P_1, \dots, P_k$

Retriever: $\text{retriever}(\mathcal{D}, Q) \rightarrow P_1, \dots, P_k$, where $k \in \mathbb{N}$ is pre-defined (e.g., 100)

Reader: $\text{reader}(Q, \{P_1, \dots, P_k\}) \rightarrow A$, similar to reading comprehension

An early retriever-reader system is **DrQA** [Chen et al., 2017]:

Retriever: a standard, “classic” TF-IDF information retrieval module (fixed)

Reader: a neural reading comprehension model, trained on SQuAD via distant supervision (i.e., by using retrieved paragraphs rather than gold ones)

Retrievers

Retrievers

Goal: find a small subset of elements (e.g., documents, paragraphs) in a datastore that are the most **similar/related/relevant** to the query

$sim(Q, P)$: similarity score between a query Q and a paragraph P

Sparse Retrieval

Similarity Score: **TF-IDF**

Sparse representation over vectors representing vocabulary/terms

Term frequency

$$\text{TF}(w, Q) = \text{count}(w, Q)$$

Inverse Document
Frequency

$$\text{IDF}(w, \mathcal{D}) = \log \frac{|\mathcal{D}|}{|\text{document freq of } w|}$$

$$q_w = \text{TF}(w, Q) \cdot \text{IDF}(w, \mathcal{D})$$

$$p_w = \text{TF}(w, P) \cdot \text{IDF}(w, \mathcal{D})$$

$$\text{sim}(Q_i, P_j) = \text{cosine}(\mathbf{q}, \mathbf{p}) \text{ with } \mathbf{q}, \mathbf{p} \in \mathbb{R}^{|V|}$$

Dense Retrieval

Similarity Score:

What if query words and document words do not match?
Dense representation over embeddings

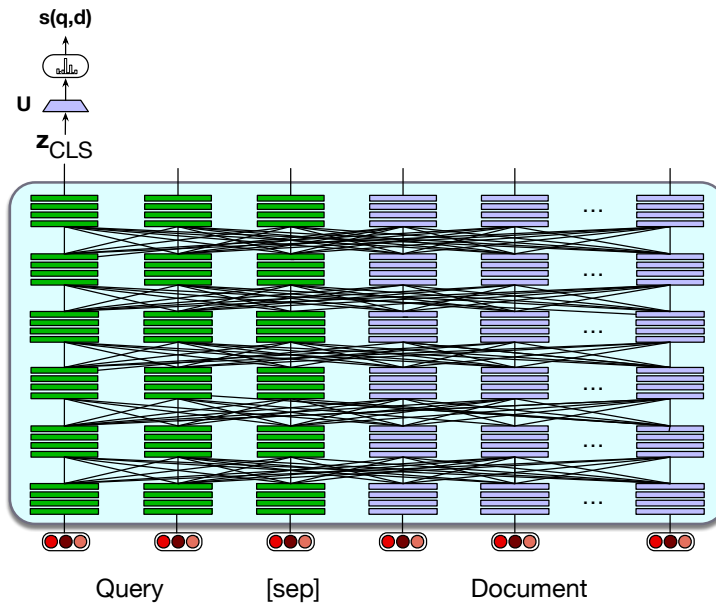
$$\text{sim}(Q_i, P_j) = \mathbf{q}_i^\top \mathbf{p}_j \text{ with } \mathbf{q}_i, \mathbf{p}_j \in \mathbb{R}^d$$

$$\mathbf{q}_i = \text{Encode}(Q_i)$$

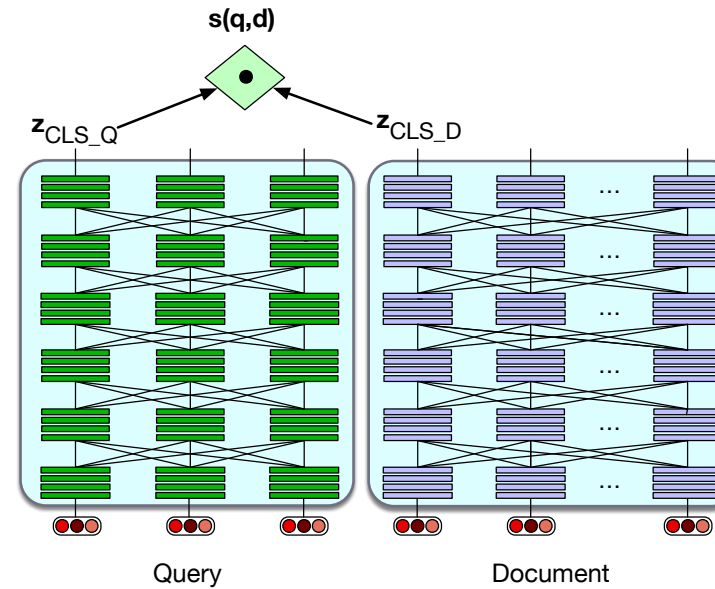
$$\mathbf{p}_j = \text{Encode}(P_j)$$

Entire research on how to improve or
learn the similarity function!

Efficient Encoding



Single encoder jointly encode query and document and finetune to produce a relevance score



Separate encoders for query and document, use dot product as score

Figure from Jurafsky and Martin (2025). Speech and Language Processing, Chapter 11

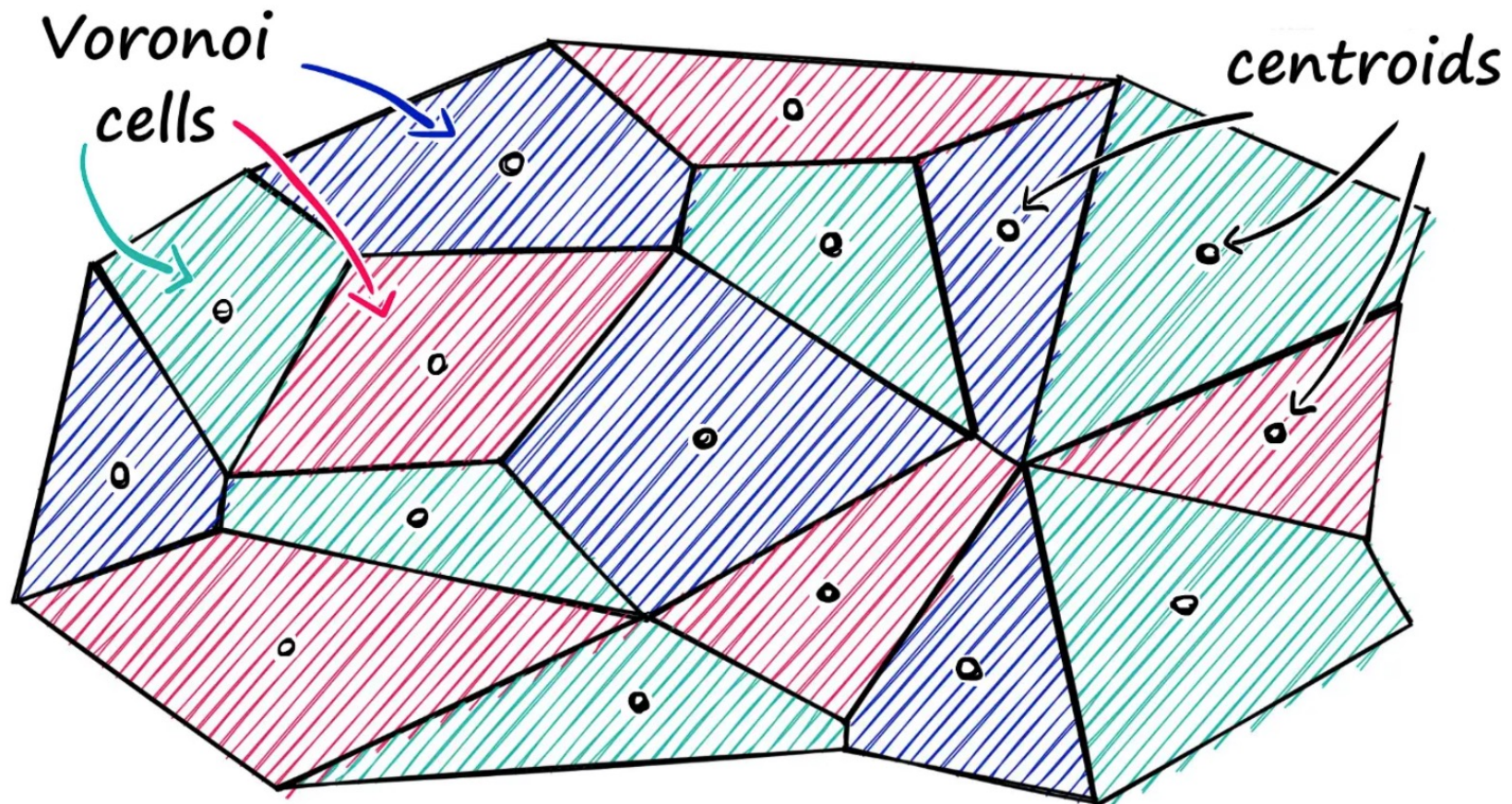
Efficient Search

Ranking every possible document according to similarity:

- **Maximum Inner Product Search (MIPS)** Uses an inverted file index (IVF) and k-means to cluster document embeddings into centroids – avoid exhaustive dot product computation
- **Facebook AI similarity search (FAISS)** A practical library to solve MIPS at scale

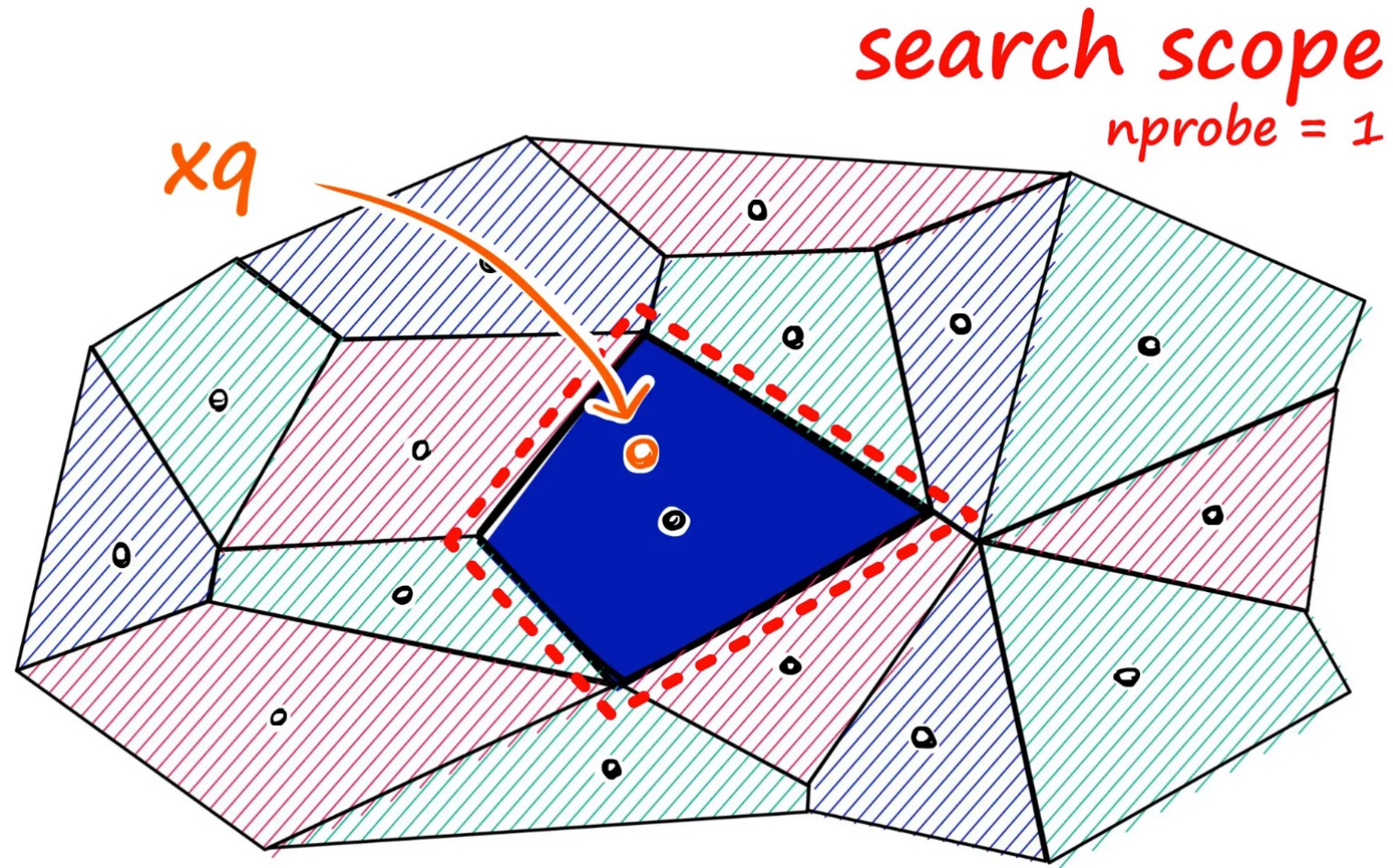
Allows dense retrieval over billions of document embeddings – making large scale RAG feasible

Efficient Search



Figures from <https://www.pinecone.io/learn/series/faiss/faiss-tutorial/>

Efficient Search: FAISS



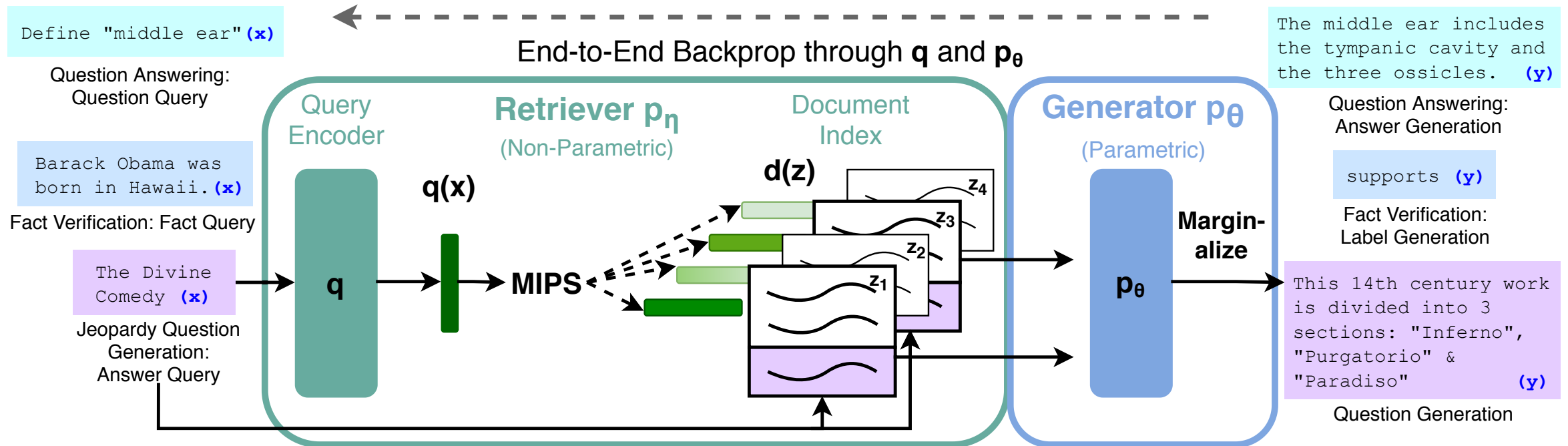
x_q – query vector

Approximate answer – can set $nprobe$ to 8 and search neighbouring cells

Figures from <https://www.pinecone.io/learn/series/faiss/faiss-tutorial/>

RAG

RAG



Innovations:

- End-to-end trainable retrieval + generation - backpropagate through marginalization
- Uses dense retrieval rather than sparse - pioneering
- Combines multiple retrieved documents during generation: probabilistic mixture

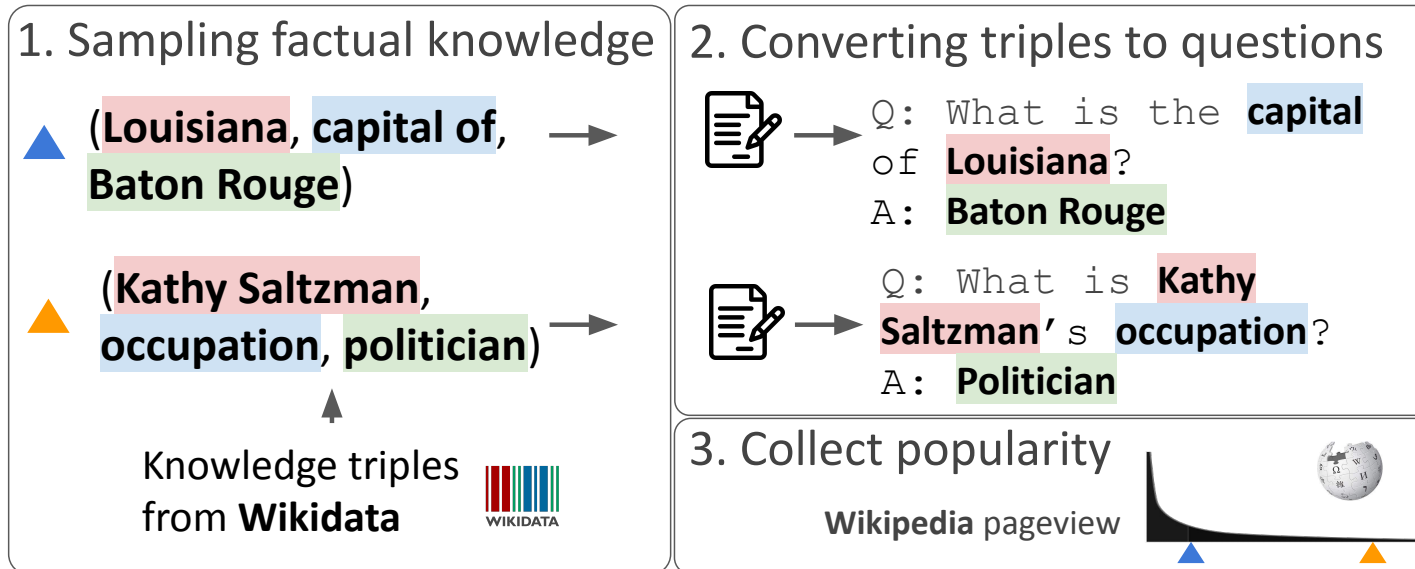
Patrick Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

RAG: Increases performance

Model		NQ
Closed Book	T5-11B [52]	34.5
	T5-11B+SSM[52]	36.6
Open Book	REALM [20]	40.4
	DPR [26]	41.5
RAG-Token		44.1
RAG-Seq.		44.5

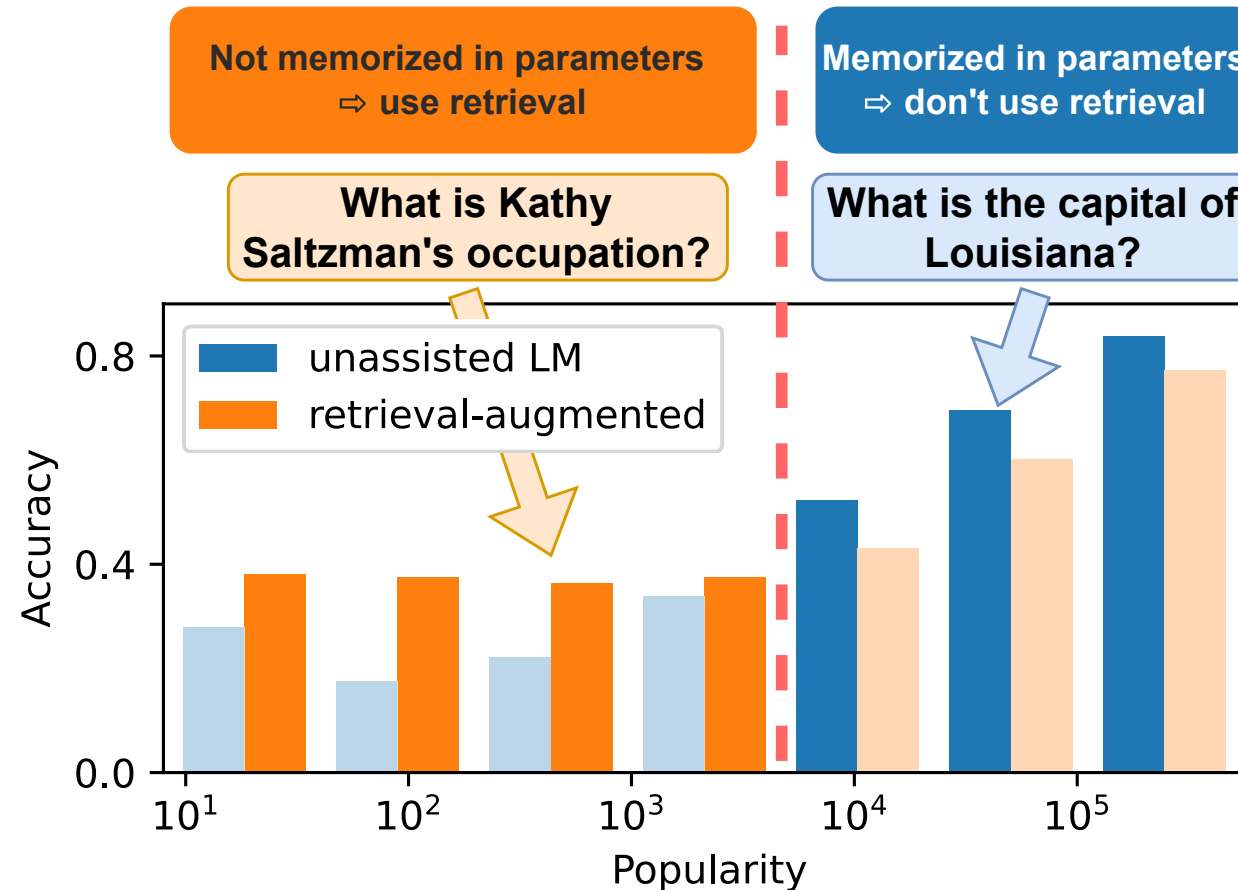
Patrick Lewis et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks

RAG: Reduces Hallucination



POPQA: sampling knowledge triples from Wikidata and converting to questions + popularity calculation based on Wikipedia page views

RAG: Reduces Hallucination for the long tail



Mallen et al. (2023). When Not to Trust Language Models: Investigating Effectiveness of Parametric and Non-Parametric Memories ACL

RAG: Better Attribution

Attribution: Ensure that generated content can be traced to **verifiable sources**

Direct Generated Attribution	 What type of herbicides are effective against sowthistle? Several herbicides can be effective against sowthistle, depending on the setting and stage of growth^[1]. Here's a list of some common herbicides that can be used: ... [1] Seber et al. xxx, Journal of Environmental Horticulture" in 2008 
Post- Retrieval Answering	 What type of herbicides are effective against sowthistle? Clopyralid and Triclopyr: Selective herbicides containing either clopyralid or triclopyr can effectively target sowthistle without harming surrounding plants. Some brand examples include Thistlex, Vivendi, Grazon Pro, and Pas-Tor¹. https://www.wikihow.com/Get-Rid-of-Thistles  
Post- Generation Attribution	 What type of herbicides are effective against sowthistle? Several herbicides can be effective against sowthistle ... Several herbicides can be...Velocity Herbicide: Particularly in the early growth stages of sowthistle, applying Velocity herbicide post-emergence is a preferred option for control¹. https://www.crop.bayer.com.au/pests/weeds/sowthistle   

Citations can be hallucinated

Most reliable but not fool proof. Widely used eg. Perplexity

Eg. Bing Chat 2

Li et al. (2023). A Survey of Large Language Models Attribution

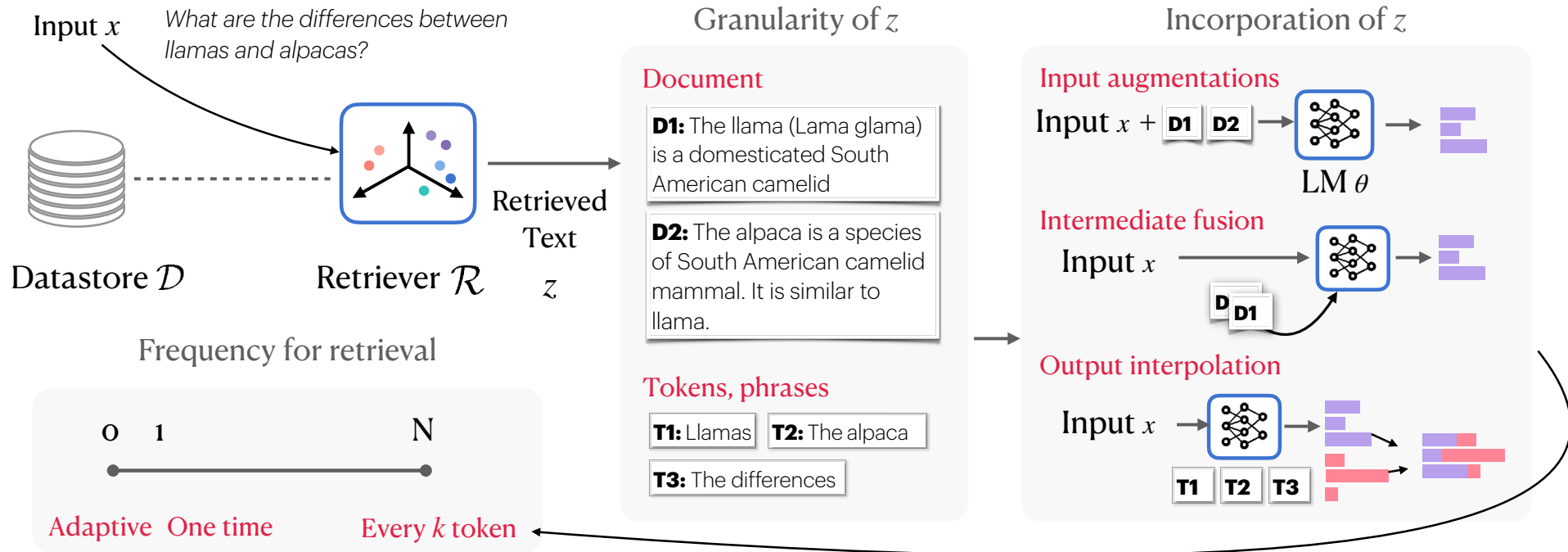
Attribution Evaluation

whether the generated statement is fully supported by the cited reference



Yue et al. (2023). Automatic Evaluation of Attribution by Large Language Models

Current Taxonomy of RAG



- what the unit of retrieved text z is (**granularity of z**),
- how z is incorporated (**incorporation of z**),
- and how often z is retrieved (**frequency of retrieval**)

Figure from Akari Asai et al. (2025). Beyond Scaling: Frontiers of Retrieval-Augmented LMs, PhD Thesis

Timeline

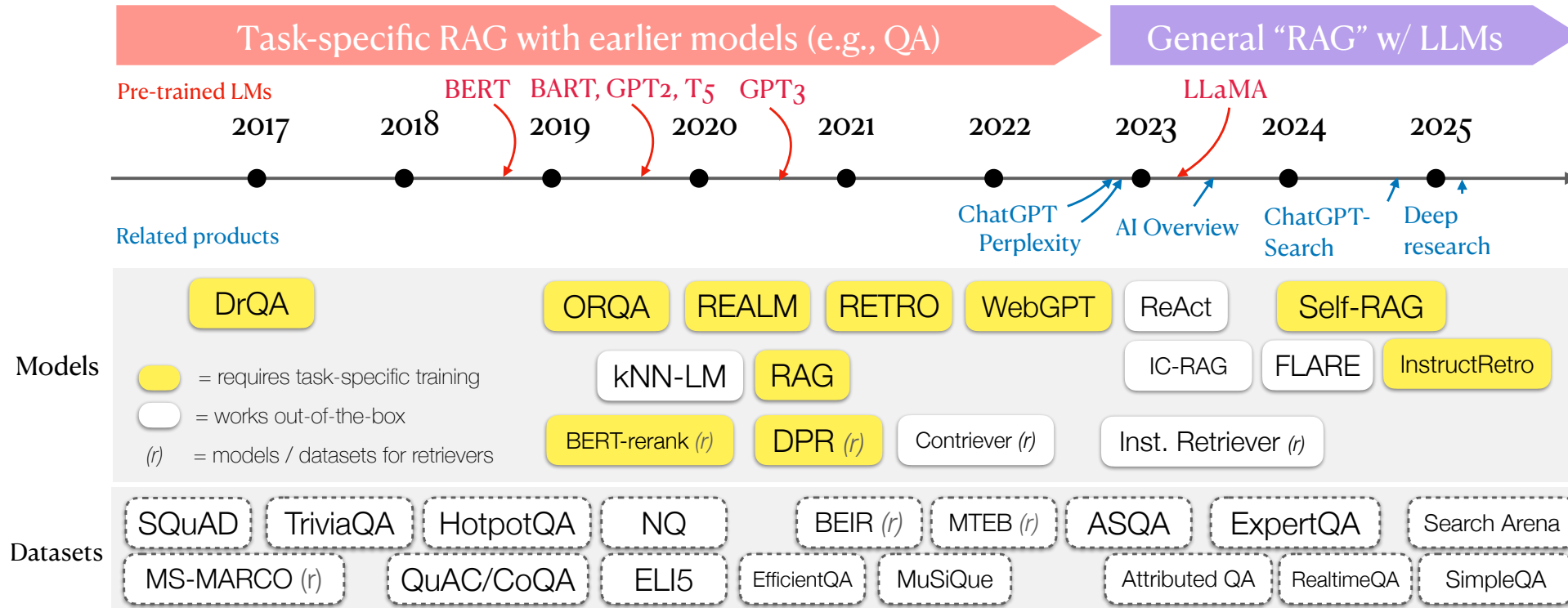
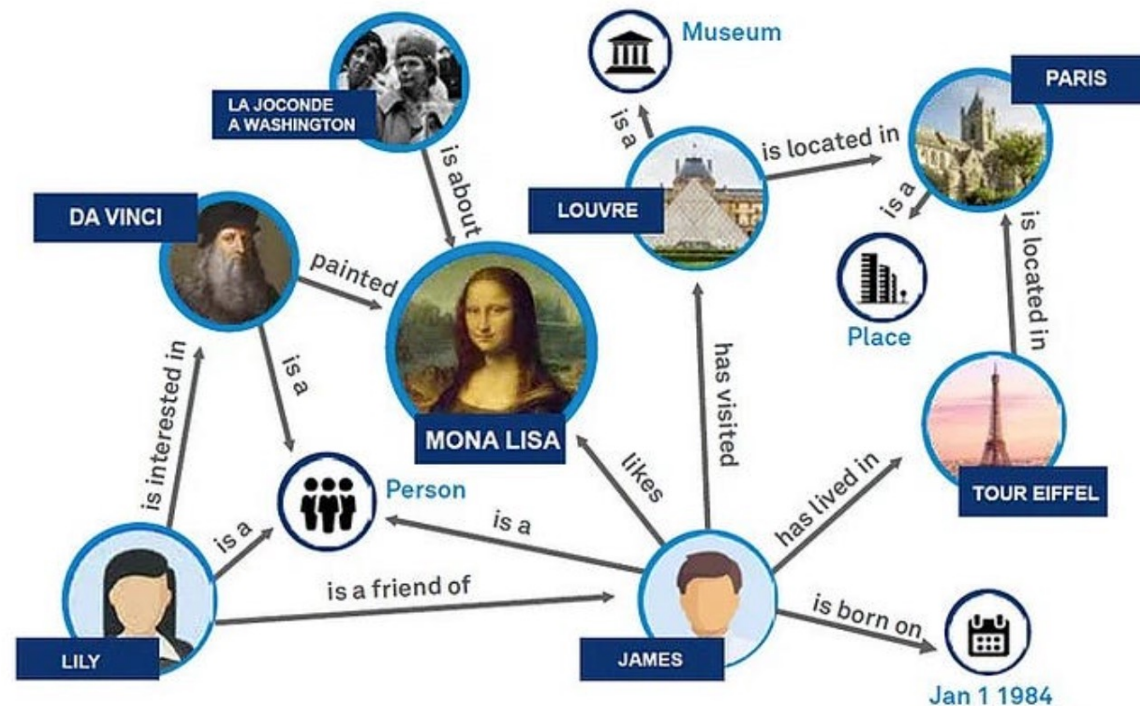


Figure from Akari Asai et al. (2025). Beyond Scaling: Frontiers of Retrieval-Augmented LMs, PhD Thesis

Advanced RAG

GraphRAG



Graph RAG Pipeline:

Knowledge Graph: Stores entities and their relationships as nodes and edges.

Graph Retriever: Traverses the graph to find relevant nodes, paths, and multi-hop connections.

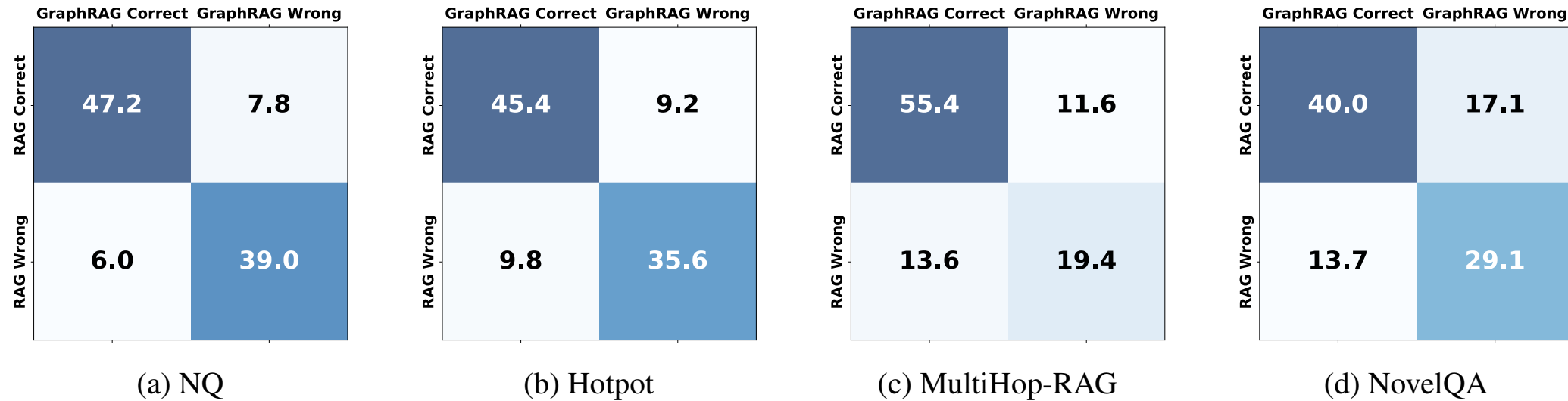
LLM: Synthesizes a response using both entities and their relationships, often providing reasoning chains.

Figure from <https://blog.langchain.com/enhancing-rag-based-applications-accuracy-by-constructing-and-leveraging-knowledge-graphs/>

GraphRAG vs RAG

- Domain requires multi-hop factual reasoning
 - Eg. “A relates to B relates to C — what does that imply?”
 - It retrieves an entire *subgraph*, capturing multiple related facts, not isolated potentially irrelevant text chunks.
- Highly interconnected data
 - Medical ontologies, enterprise knowledge
 - It can navigate edges like “*is part of*”, “*depends on*”, “*caused by*”, “*parent of*”, etc., which are essential context.
- When you need summaries across documents
 - Eg. “Explain the connection between these two policies.” or “Give me all information about product X and its dependencies.”
 - It can retrieve connected components and clusters – higher level summaries
- Retrieval can scale to millions of nodes with structured queries
 - When knowledge bases are too large for chunk-based RAG to stay relevant or efficient.
 - Graph traversal is cheap and retrieval is more relevant than dense-vector similarity alone

Performance GraphRAG



RAG excels single-hop, detail oriented retrieval

GraphRAG shines in multi-hop retrieval and global summarisation

Han et al. (2025) RAG vs. GraphRAG: A Systematic Evaluation and Key Insights

Agentic RAG

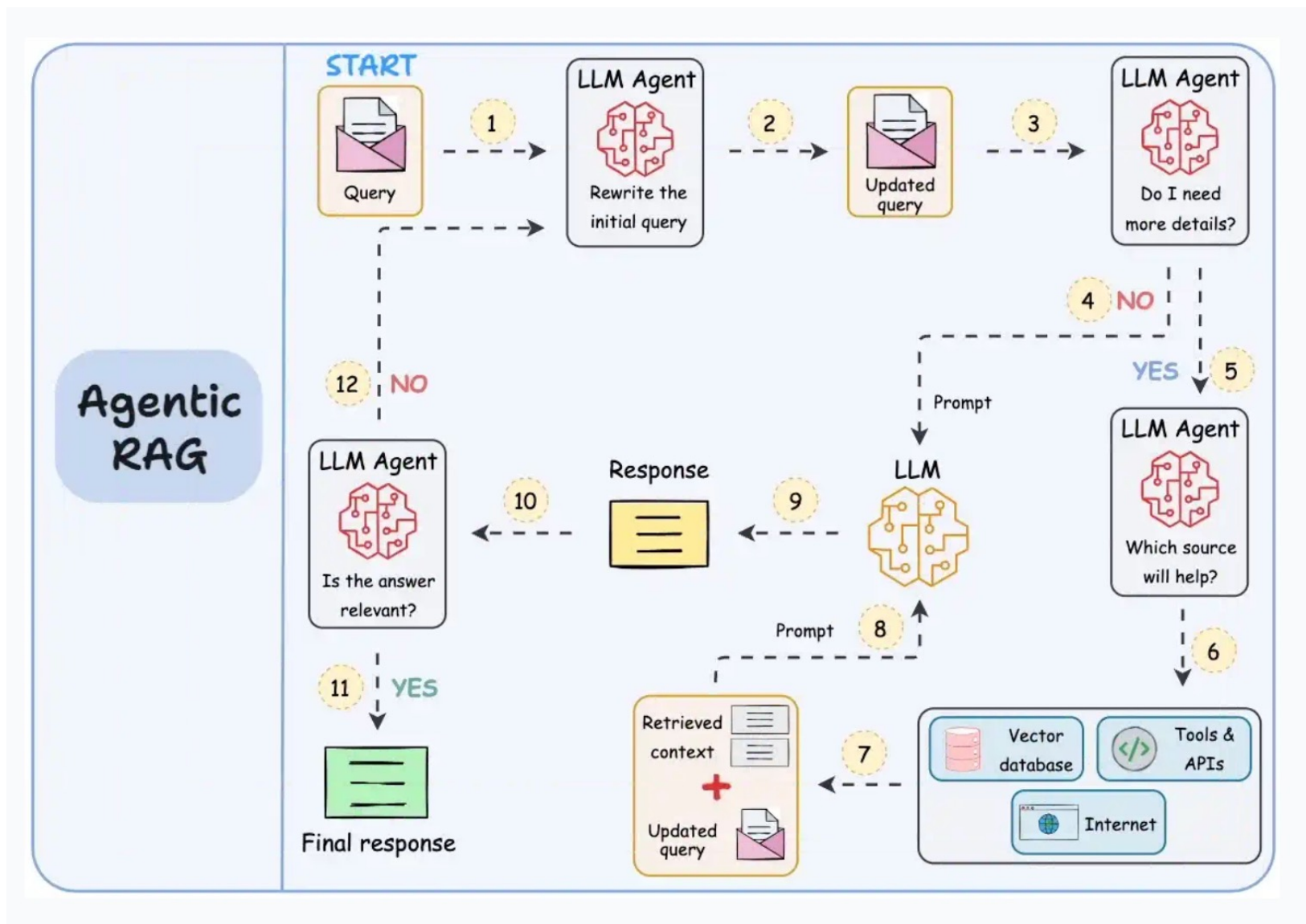
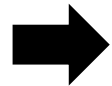


Figure from <https://blog.dailydoseofds.com/p/rag-vs-agentic-rag>

Summary

- Limitations of LLM knowledge
- Retriever-Reader Framework
- Sparse and Dense Retrieval
- RAG and its advantages and limitations
- GraphRAG and Agentic RAG



Agents Lecture 28 and 29 in week 10